Intro to Data Science - Lab 9

Copyright 2025, Jeffrey Stanton and Jeffrey Saltz Please do not post online.

Week 9 - Supervised Data Mining

```
# Enter your name here: Sofia Quintero
```

Please include nice comments.

Instructions:

Run the necessary code on your own instance of R-Studio.

Attribution statement: (choose only one and delete the rest)

```
# 1. I did this lab assignment by myself, with help from the book and the professor.
```

Supervised data mining/machine learning is the most prevalent form of data mining as it allows for the prediction of new cases in the future. For example, when credit card companies are trying to detect fraud, they will create a supervised model by training it on fraud data that they already have. Then they will deploy the model into the field: As new input data arrives the model predicts whether it seems fraudulent and flags those transactions where that probability is high.

In these exercises we will work with a built-in data set called **GermanCredit**. This data set is in the ** caret ** package so we will need that and the ** kernlab ** package to be installed and libraried before running the following:

```
data("GermanCredit")
subCredit <- GermanCredit[,1:10]
str(subCredit)
```

```
install.packages('caret', repos = "http://cran.us.r-project.org")
```

```
##
## The downloaded binary packages are in
##  /var/folders/rm/s4vv52kd55g34xzsxxqn9p2c0000gn/T//Rtmpj7WMgL/downloaded_packages
```

```
install.packages('kernlab', repos = "http://cran.us.r-project.org")
```

```
##
## The downloaded binary packages are in
##  /var/folders/rm/s4vv52kd55g34xzsxxqn9p2c0000gn/T//Rtmpj7WMgL/downloaded_packages
```

```
library('caret')
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```r
library('kernlab')
```

```
##
## Attaching package: 'kernlab'

## The following object is masked from 'package:ggplot2':
##
##     alpha
```

```r
data("GermanCredit")
subCredit <- GermanCredit[,1:10]
str(subCredit)
```

```
## 'data.frame':    1000 obs. of  10 variables:
##  $ Duration                 : int  6 48 12 42 24 36 24 36 12 30 ...
##  $ Amount                   : int  1169 5951 2096 7882 4870 9055 2835 6948 3059 5234 ...
##  $ InstallmentRatePercentage: int  4 2 2 2 3 2 3 2 2 4 ...
##  $ ResidenceDuration        : int  4 2 3 4 4 4 4 2 4 2 ...
##  $ Age                      : int  67 22 49 45 53 35 53 35 61 28 ...
##  $ NumberExistingCredits    : int  2 1 1 1 2 1 1 1 1 2 ...
##  $ NumberPeopleMaintenance  : int  1 1 2 2 2 2 1 1 1 1 ...
##  $ Telephone                : num  0 1 1 1 1 0 1 0 1 1 ...
##  $ ForeignWorker            : num  1 1 1 1 1 1 1 1 1 1 ...
##  $ Class                    : Factor w/ 2 levels "Bad","Good": 2 1 2 2 1 2 2 2 2 1 ...
```

1. Examine the data structure that **str()** reveals. Also use the **help()** command to learn more about the **GermanCredit** data set. Summarize what you see in a comment.

```r
help("GermanCredit")

#str() reveals that GermanCredit has 1000 rows and 10 columns.
#the int type variables are: duration, amount, installment rate percentage,
#residence duration, age, existing credits, and people maintenance.
#the num variables are: telephone and foreign worker
#class includes "good" or "bad" represented by 1 and 2.
#many of the predictors are expended into 0/1 indicator variables.
```

2. Use the **createDataPartition()** function to generate a list of cases to include in the training data. This function is conveniently provided by caret and allows one to directly control the number of training cases. It also ensures that the training cases are balanced with respect to the outcome variable. Try this: `trainList <- createDataPartition(y=subCredit$Class,p=.40,list=FALSE)`

```r
trainList <- createDataPartition(y=subCredit$Class,p=.40,list=FALSE)
```

3. Examine the contents of **trainList** to make sure that it is a list of case numbers. With **p=0.40**, it should have 400 case numbers in it.

```r
str(trainList)
```

```
##  int [1:400, 1] 2 7 10 11 12 13 14 18 19 21 ...
##  - attr(*, "dimnames")=List of 2
##   ..$ : NULL
##   ..$ : chr "Resample1"
```

```r
#trainList has 400 case numbers in it
```

4. What is **trainList**? What do the elements in **trainList** represent? Which attribute is balanced in the **trainList** dataset?

```r
#trainList is a partition of data used for testing taken from the larger
#data set of German Credit. The elements represent which data values from
#the larger dataset will be used for testing. The Class attribute is balanced in
#in trainList.
```

5. Use **trainList** and the square brackets notation to create a training data set called ** trainSet ** from the **subCredit** data frame. Look at the structure of trainSet to make sure it has all of the same variables as **subCredit**. The **trainSet** structure should be a data frame with **400 rows and 10 columns**.

```r
trainSet <- subCredit[trainList,]
str(trainSet)
```

```
## 'data.frame':    400 obs. of  10 variables:
##  $ Duration                 : int  48 24 30 12 48 12 24 30 24 9 ...
##  $ Amount                   : int  5951 2835 5234 1295 4308 1567 1199 8072 12579 2134 ...
##  $ InstallmentRatePercentage: int  2 3 4 3 3 1 4 2 4 4 ...
##  $ ResidenceDuration        : int  2 4 2 1 4 1 4 3 2 4 ...
##  $ Age                      : int  22 53 28 25 24 22 60 25 44 48 ...
##  $ NumberExistingCredits    : int  1 1 2 1 1 1 2 3 1 3 ...
##  $ NumberPeopleMaintenance  : int  1 1 1 1 1 1 1 1 1 1 ...
##  $ Telephone                : num  1 1 1 1 1 0 1 1 0 0 ...
##  $ ForeignWorker            : num  1 1 1 1 1 1 1 1 1 1 ...
##  $ Class                    : Factor w/ 2 levels "Bad","Good": 1 2 1 1 1 2 1 2 1 2 ...
```

6. Use **trainList** and the square brackets notation to create a testing data set called ** testSet ** from the subCredit data frame. The **testSet** structure should be a data frame with **600 rows and 10 columns** and should be a completely different set of cases than **trainSet**.

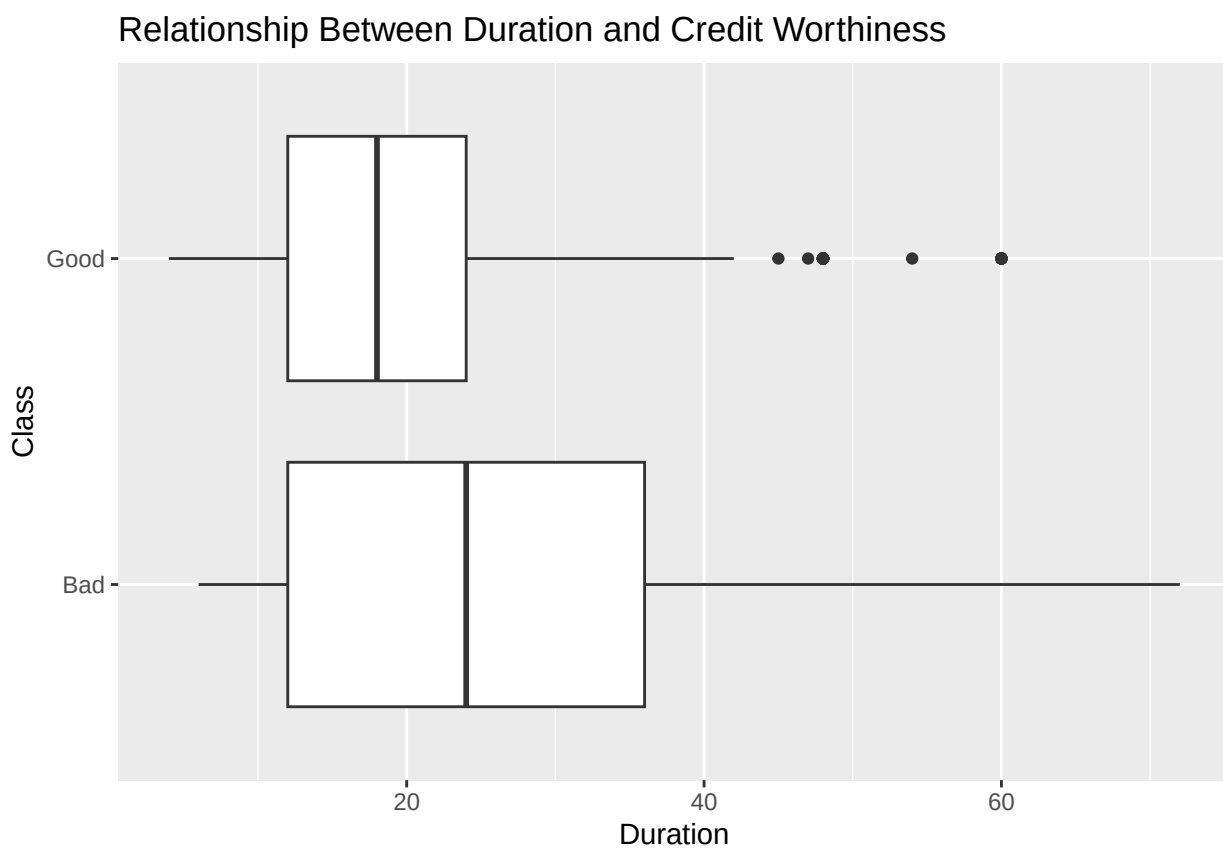
```r
testSet <- subCredit[-trainList,]
str(testSet)
```

```
## 'data.frame':    600 obs. of  10 variables:
##  $ Duration                 : int  6 12 42 24 36 36 12 15 24 24 ...
##  $ Amount                   : int  1169 2096 7882 4870 9055 6948 3059 1403 1282 2424 ...
##  $ InstallmentRatePercentage: int  4 2 2 3 2 2 2 2 4 4 ...
##  $ ResidenceDuration        : int  4 3 4 4 4 2 4 4 2 4 ...
##  $ Age                      : int  67 49 45 53 35 35 61 28 32 53 ...
```

```
##  $ NumberExistingCredits    : int  2 1 1 2 1 1 1 1 1 2 ...
##  $ NumberPeopleMaintenance  : int  1 2 2 2 2 1 1 1 1 1 ...
##  $ Telephone                : num  0 1 1 1 0 0 1 1 1 1 ...
##  $ ForeignWorker            : num  1 1 1 1 1 1 1 1 1 1 ...
##  $ Class                    : Factor w/ 2 levels "Bad","Good": 2 2 2 1 2 2 2 2 1 2 ...
```

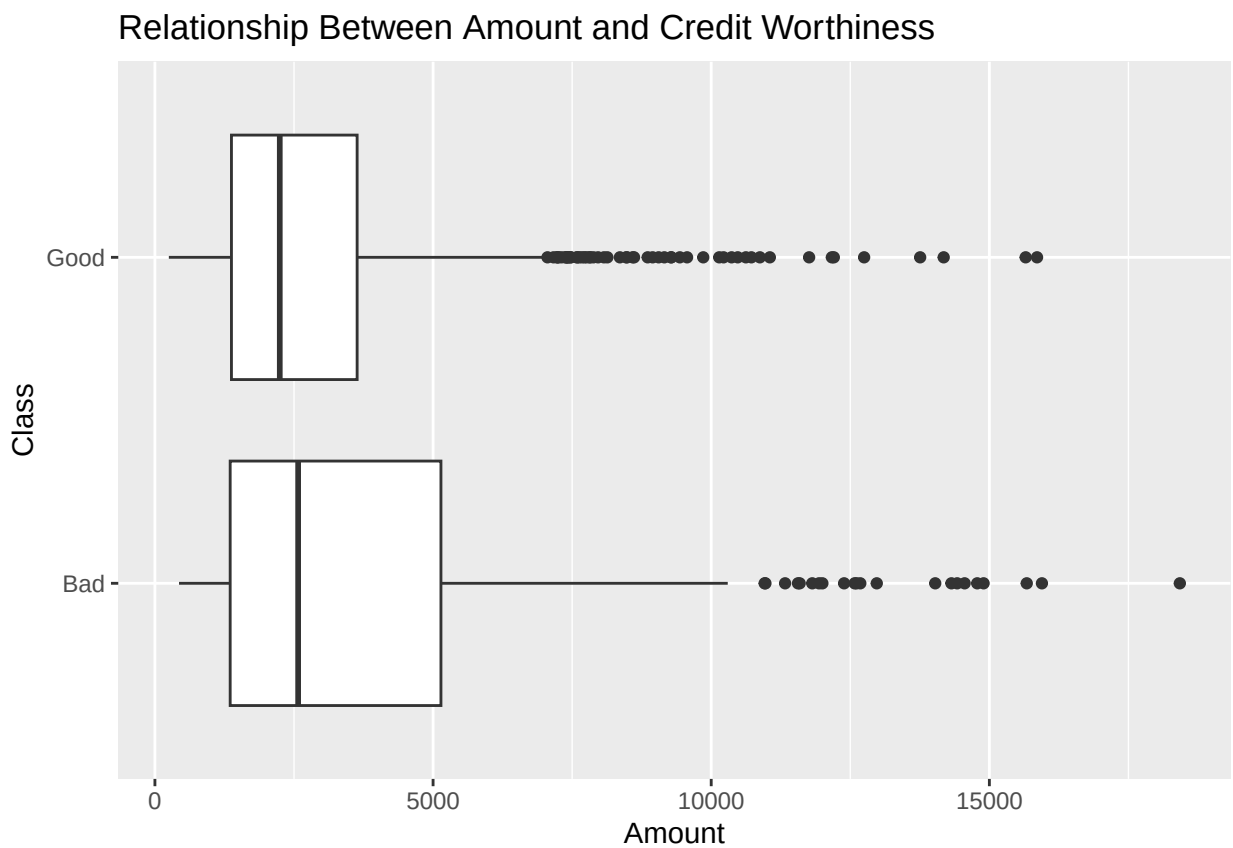
7. Create and interpret boxplots of all the predictor variables in relation to the outcome variable (**Class**).

```r
#Class and Duration
durationPlot <- ggplot(subCredit, aes(x=Duration, y=Class)) +
  geom_boxplot() + xlab("Duration") +
  ggtitle("Relationship Between Duration and Credit Worthiness")
durationPlot
```



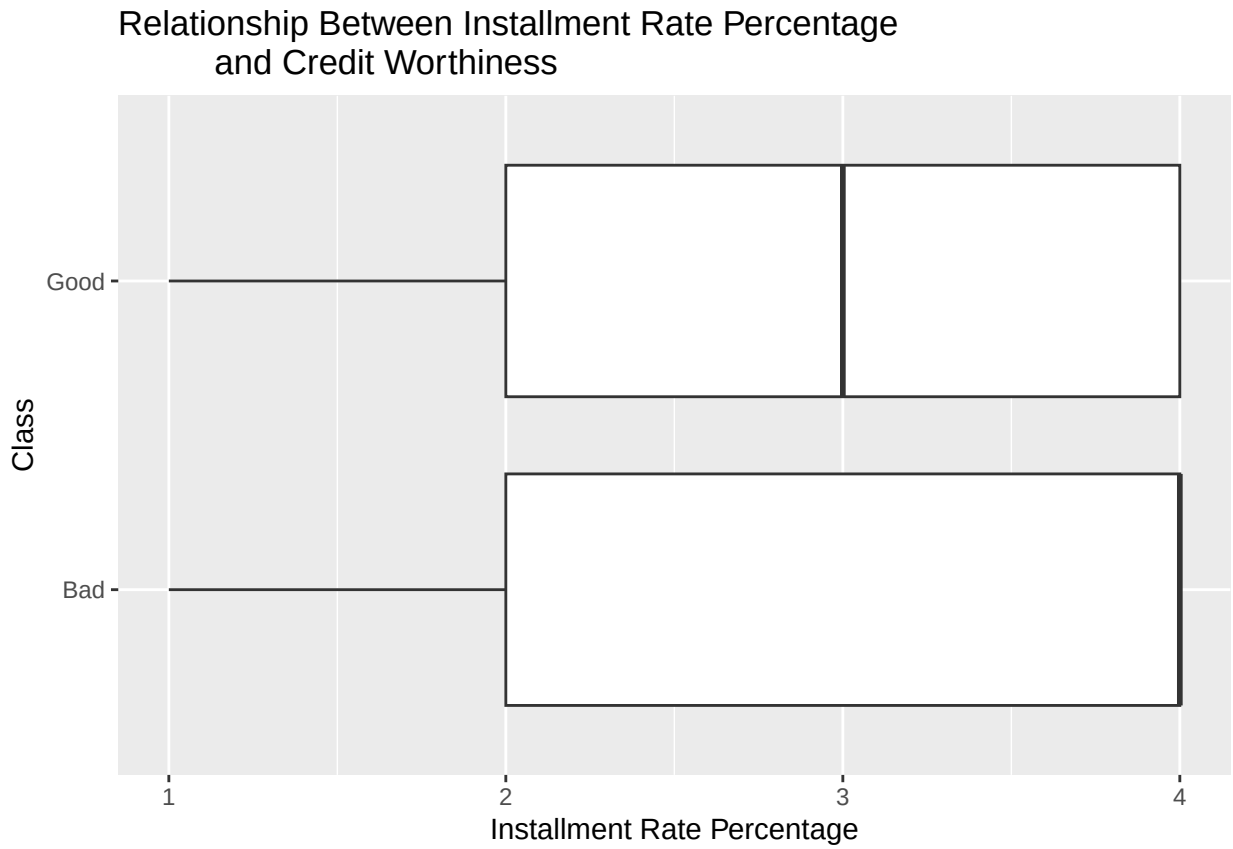

```r
#Accounts with shorter durations appear to have better credit
```

```r
#Class and Amount
amountPlot <- ggplot(subCredit, aes(x=Amount, y=Class)) +
  geom_boxplot() + xlab("Amount") +
  ggtitle("Relationship Between Amount and Credit Worthiness")
amountPlot
```



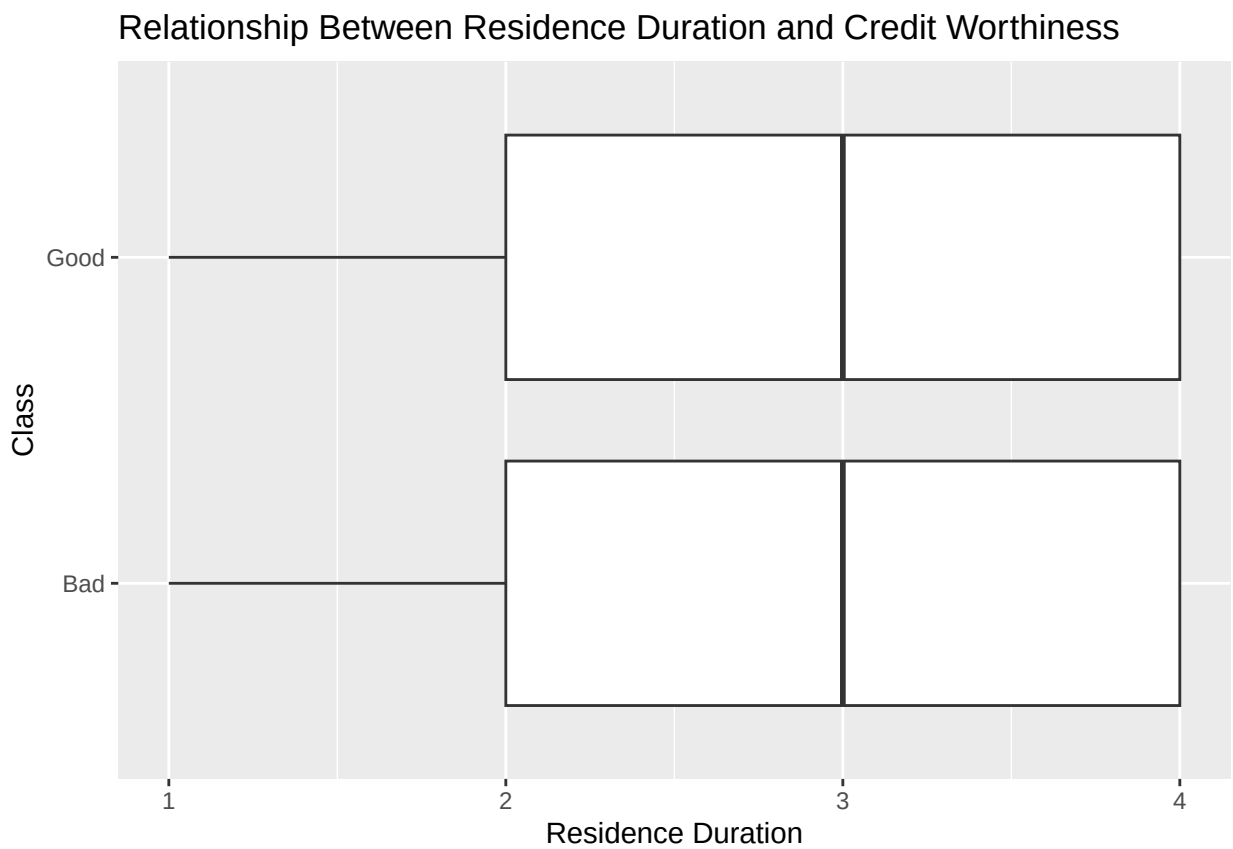

```
#the boxplots for account are roughly equal, although good credit has
#a lower upper quartile for amount. This predictor might not be as
#influential in determining class outcome.
```

```
#Class and Installment Rate Percentage
irpPlot <- ggplot(subCredit, aes(x=InstallmentRatePercentage,
                                 y=Class)) + geom_boxplot() +
  xlab("Installment Rate Percentage") +
  ggtitle("Relationship Between Installment Rate Percentage
          and Credit Worthiness")
irpPlot
```



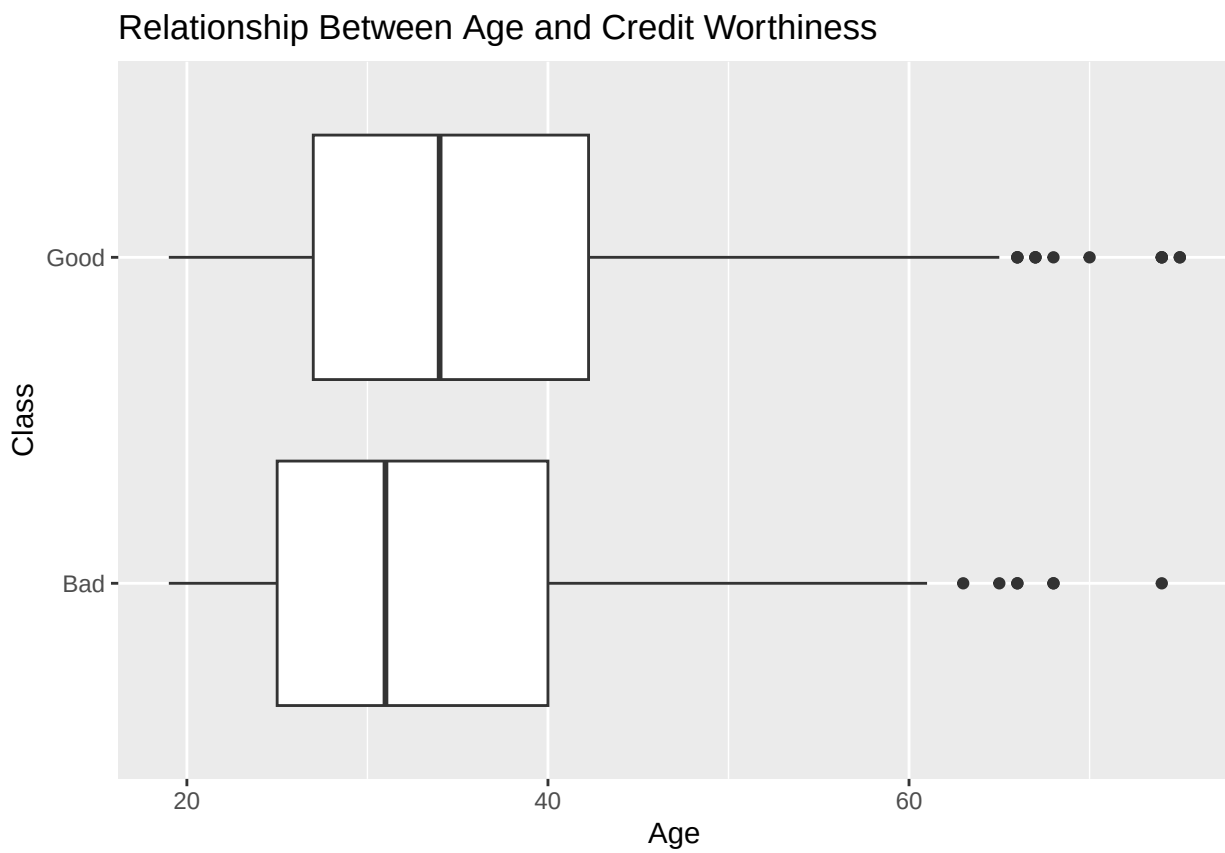

```
#the boxplots are roughly equal for installment rate percentage,
#but bad credit has a higher mean for installment rate percentage.
#this predictor appears to have minimal influence on class outcome.
```

```
#Class and Residence Duration
residencePlot <- ggplot(subCredit, aes(x=ResidenceDuration, y=Class)) +
  geom_boxplot() + xlab("Residence Duration") +
  ggtitle("Relationship Between Residence Duration and Credit Worthiness")
residencePlot
```



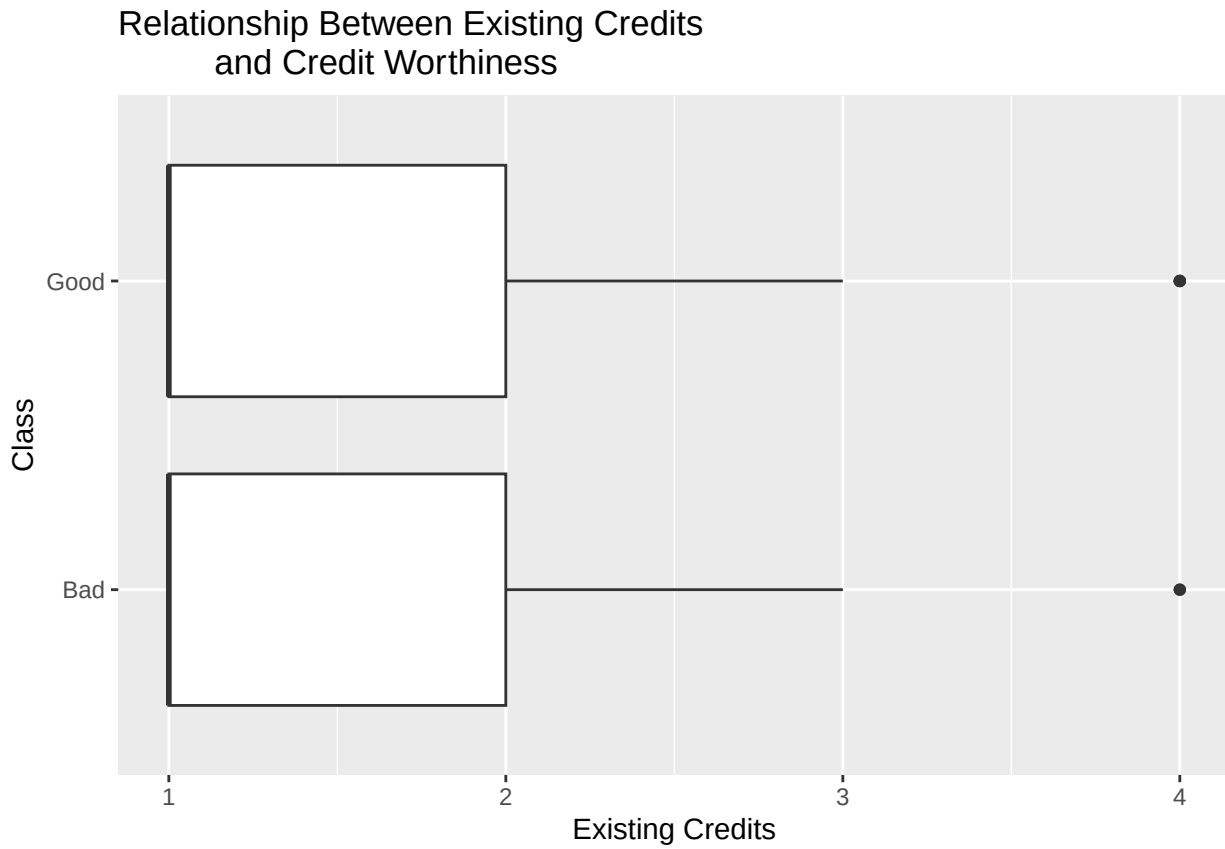

```
#the boxplots for residence duration appear to be equal, meaning this predictor
#has minimal influence on class outcome
```

```
#Class and Age
agePlot <- ggplot(subCredit, aes(x=Age, y=Class)) +
  geom_boxplot() + xlab("Age") +
  ggtitle("Relationship Between Age and Credit Worthiness")
agePlot
```



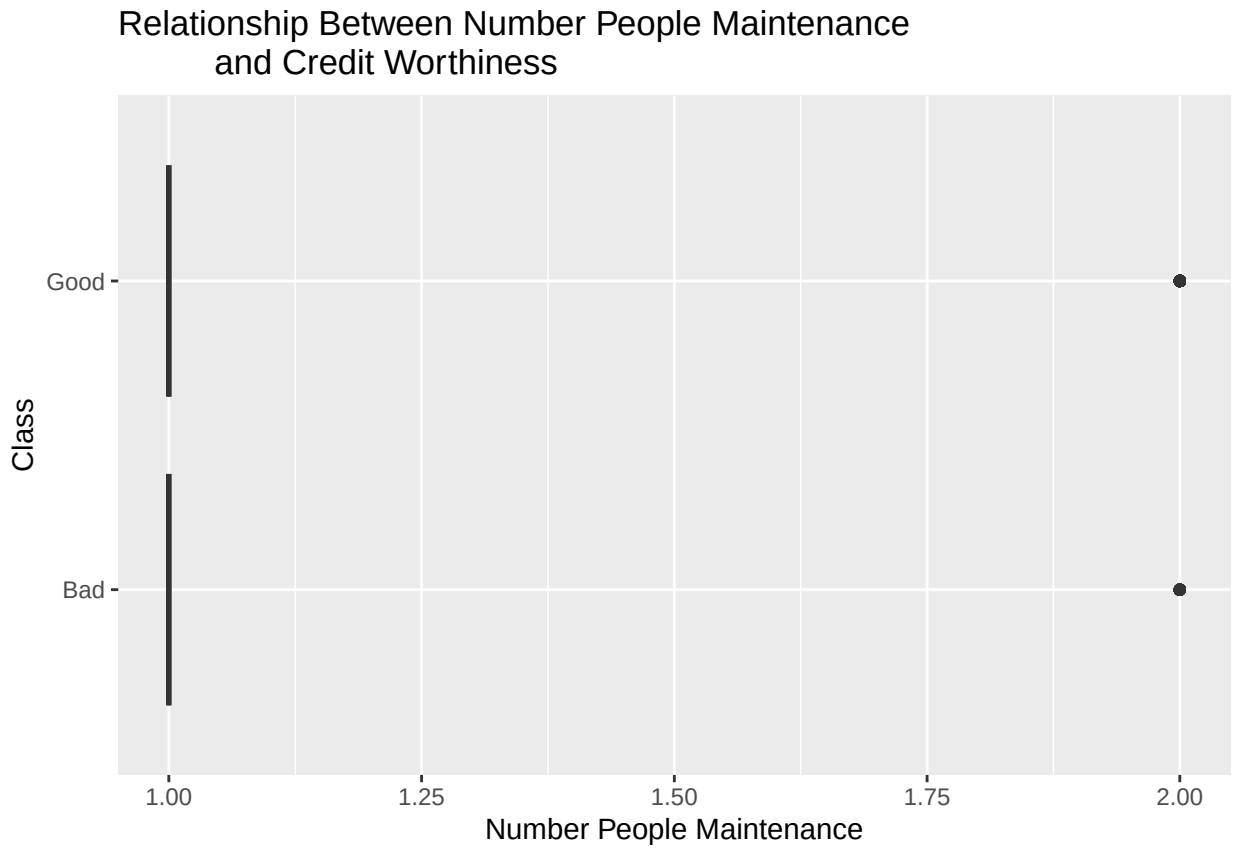

```
#the age boxplot for good credit includes slightly greater
#measures of variability, age might influence the class outcome.
```

```
#Class and Existing Credits
existingPlot <- ggplot(subCredit,
                       aes(x=NumberExistingCredits, y=Class)) +
  geom_boxplot() + xlab("Existing Credits") +
  ggtitle("Relationship Between Existing Credits
          and Credit Worthiness")
existingPlot
```



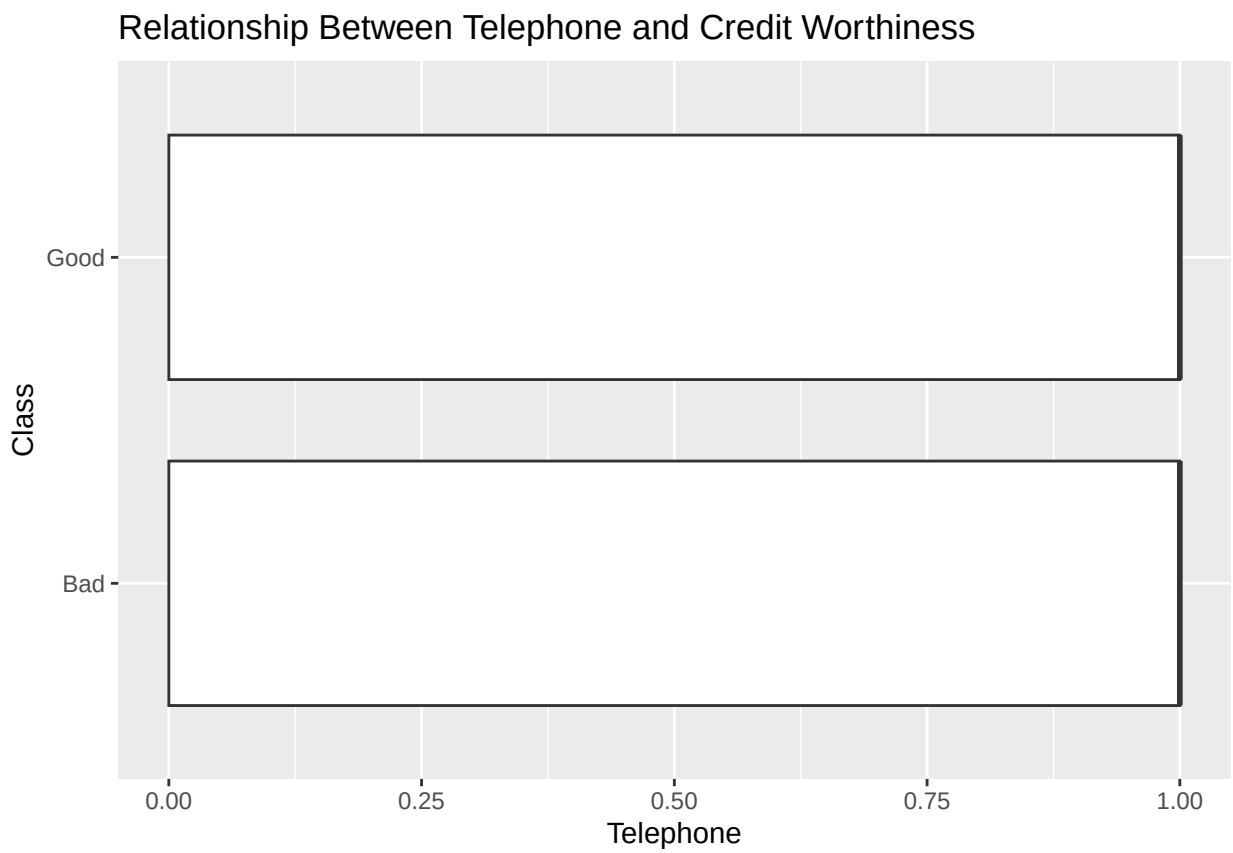

```
#the boxplots for number of exisiting credits appear equal,
#meaning this predictor has minimal influence on class.
```

```
#Class and People Maintenance
maintenancePlot <- ggplot(subCredit,
                          aes(x=NumberPeopleMaintenance, y=Class)) +
  geom_boxplot() + xlab("Number People Maintenance") +
  ggtitle("Relationship Between Number People Maintenance
          and Credit Worthiness")
maintenancePlot
```



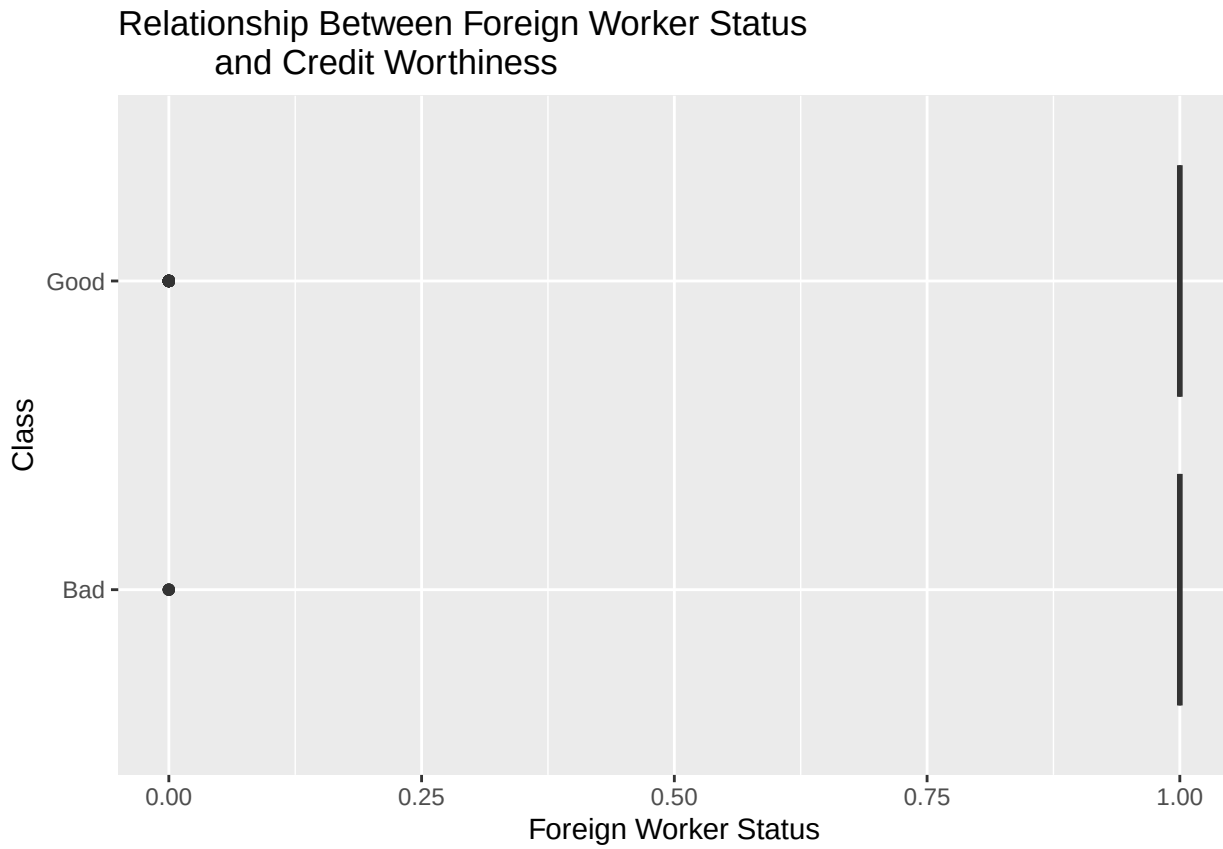

```
#the boxplots for people maintenance appear equal,
#meaning this predictor has minimal influence on class.
```

```
#Class and Telephone
telephonePlot <- ggplot(subCredit, aes(x=Telephone, y=Class)) +
  geom_boxplot() + xlab("Telephone") +
  ggtitle("Relationship Between Telephone and Credit Worthiness")
telephonePlot
```




```
#the boxplots for telephone appear equal, meaning this predictor
#has minimal influence on class.
```

```
#Class and Foreign Worker Status
foreignPlot <- ggplot(subCredit, aes(x=ForeignWorker, y=Class)) +
  geom_boxplot() + xlab("Foreign Worker Status") +
  ggtitle("Relationship Between Foreign Worker Status
          and Credit Worthiness")
foreignPlot
```




```
#the boxplots for foreign status appear equal, meaning this
#predictor has minimal influence on class.
```

8. Train a support vector machine with the **ksvm()** function from the **kernlab** package. Make sure that you have installed and libraried the **kernlab** package. Have the **cost** be 5, and have **ksvm** do 3 **cross validations** (Hint: `try prob.model = TRUE`)

```
svmOut <- ksvm(Class ~ ., data=trainSet, kernel= "rbfdot", kpar = "automatic",
     C = 5, cross = 3, prob.model = TRUE)
svmOut
```

```
## Support Vector Machine object of class "ksvm"
##
## SV type: C-svc  (classification)
##  parameter : cost C = 5
##
## Gaussian Radial Basis kernel function.
##  Hyperparameter : sigma =  0.103037213209918
##
## Number of Support Vectors : 250
##
## Objective Function Value : -835.4804
## Training error : 0.14
## Cross validation error : 0.302566
## Probability model included.
```

9. Examine the ksvm output object. In particular, look at the **cross-validation error** for an initial indication of model quality. Add a comment that gives your opinion on whether this is a good model.

```
#The cross validation error is 0.29746. This indicates the model is approximately
#70% accurate
```

10. Predict the training cases using the **predict()** command

```
svmPred <- predict(svmOut, testSet, type = "response")
```

11. Examine the predicted out object with **str( )**. Then, calculate a **confusion matrix** using the **table()** function.

```
str(svmPred) #it appears majority of the data
```

```
##  Factor w/ 2 levels "Bad","Good": 2 1 1 2 2 1 2 2 2 2 ...
```

```
#returns as 2 ("Good")
confMatrx <- table(testSet$Class,svmPred)
confMatrx
```

```
##       svmPred
##        Bad Good
##   Bad   47  133
##   Good  60  360
```

12. Interpret the confusion matrix and in particular calculate the overall **accuracy** of the model. The **diag( )** command can be applied to the results of the table command you ran in the previous step. You can also use **sum( )** to get the total of all four cells.

```
#The matrix calculated more false positives than false negatives

diag(confMatrx)/sum(confMatrx)
```

```
##        Bad       Good
## 0.07833333 0.60000000
```

```
sum(diag(confMatrx))/sum(confMatrx)
```

```
## [1] 0.6783333
```

13. Check you calculation with the **confusionMatrix()** function in the **caret** package.

```
confusionMatrix(testSet$Class, svmPred)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Bad Good
```

```
##       Bad   47  133
##       Good  60  360
##
##                Accuracy : 0.6783
##                  95% CI : (0.6393, 0.7156)
##     No Information Rate : 0.8217
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.1338
##
##  Mcnemar's Test P-Value : 2.187e-07
##
##             Sensitivity : 0.43925
##             Specificity : 0.73022
##          Pos Pred Value : 0.26111
##          Neg Pred Value : 0.85714
##              Prevalence : 0.17833
##          Detection Rate : 0.07833
##    Detection Prevalence : 0.30000
##       Balanced Accuracy : 0.58474
##
##        'Positive' Class : Bad
##
```

```
#The accuracy calculation was correct
```